

Experiment No.	1
Experiment Title	Working with Python packages – NumPy, SciPy, Scikit-Learn, Matplotlib ¹
Student Name	Simiyon Vinscent Samuel L
Register Number	3122247001062
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

To explore the core functions of NumPy, Pandas, SciPy, Scikit-learn and Matplotlib, and to survey five reference datasets in order to identify the type of machine learning task (supervised/unsupervised, classification/regression), a suitable feature-selection technique, and a suitable algorithm for each.

2 Problem Statement

This experiment has two parts. Part A explores the core functions of five Python data-science libraries (NumPy, Pandas, SciPy, Scikit-learn, Matplotlib) that the rest of this laboratory course builds on. Part B surveys five reference datasets (Iris, Loan Amount Prediction, Pima Indians Diabetes, Spambase, and the **digits** handwritten-character dataset) to identify, for each, the appropriate machine learning task type (supervised/unsupervised, classification/regression), a suitable feature-selection technique, and a suitable algorithm family. Unlike Experiments 2–4, no single model is trained or tuned here; the goal is a working familiarity with the tooling and a taxonomy of techniques that motivates the modelling choices made in the later experiments.

3 Dataset Description

Dataset	Samples	Features	Classes / Target
Iris	150	4 numeric	3 classes (species)
Loan Amount Prediction	614 applications (149 missing values across all columns)	5 numeric + 5 categorical	Continuous target (LoanAmount)
Pima Indians Diabetes	768	8 numeric	2 classes (500 negative / 268 positive)
Spambase	4601 emails	57 numeric, 0 missing	2 classes (spam / ham)
digits (MNIST stand-in)	1797	64 (8×8 pixel intensities)	10 classes (digits 0–9)

¹GitHub Repository: <https://github.com/blackscythe123/Machine-Learning-course>

4 Exploratory Data Analysis

Iris Dataset

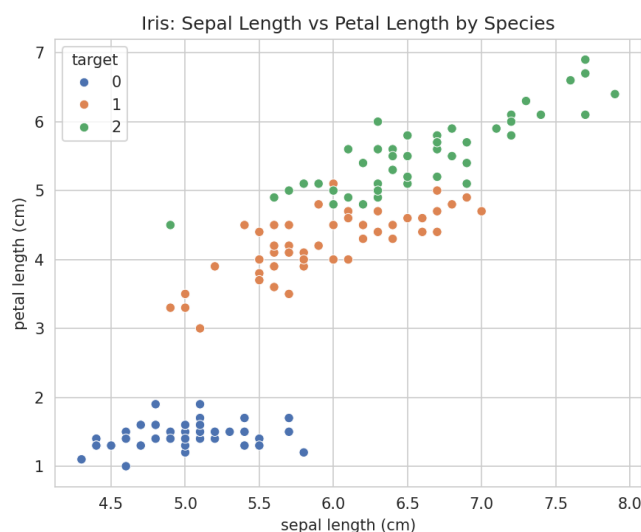


Figure 1: Iris: sepal length vs. petal length, coloured by species (150 samples, 4 features, 3 classes).

Inference: *Setosa* is linearly separable from the other two species on this feature pair alone; *versicolor* and *virginica* overlap slightly, which is why simple linear classifiers already achieve high accuracy on Iris but a non-linear boundary helps at the versicolor/virginica margin.

Loan Amount Prediction

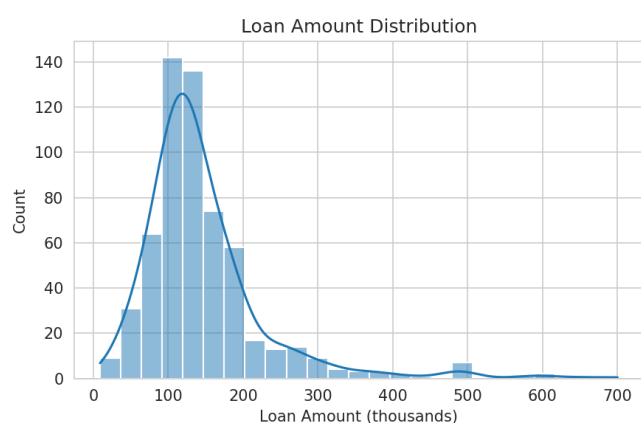


Figure 2: Distribution of `LoanAmount` (614 applications, 149 missing values across all columns).

Inference: The right-skewed target and the mix of 5 numeric + 5 categorical features (with real missing values) make this a regression task requiring both imputation and categorical encoding, which is exactly the pipeline built in Experiment 3.

Predicting Diabetes

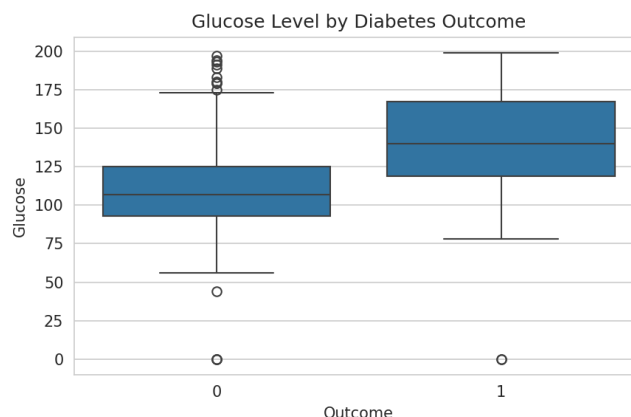


Figure 3: Glucose level by diabetes outcome (Pima Indians Diabetes dataset, 768 samples, class balance 500 negative / 268 positive).

Inference: Glucose level shows a clear separation between the two outcome classes (higher median and higher spread for the diabetic group), making it, along with BMI and Age, one of the strongest univariate predictors identified by the ANOVA F-test.

Classification of Email Spam

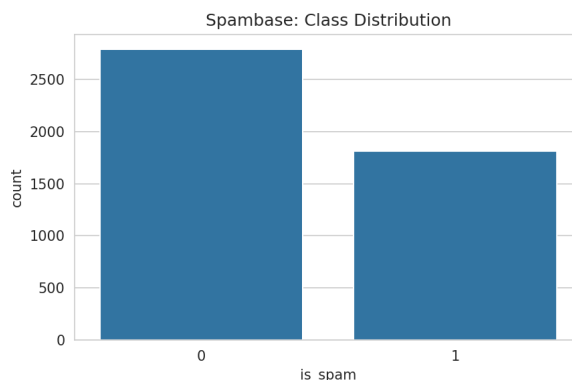


Figure 4: Spambase class distribution (4601 emails, 57 numeric features, 0 missing values).

Inference: This is a moderately imbalanced binary classification task on purely numeric, non-negative frequency features (the same dataset used and analyzed in depth in Experiments 2 and 4).

Handwritten Character Recognition / MNIST

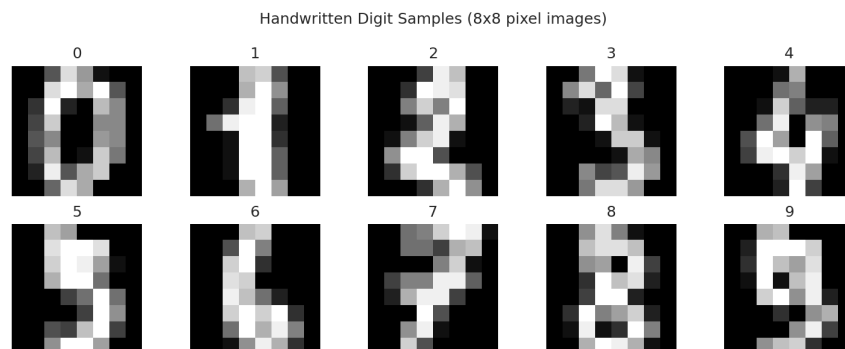


Figure 5: Sample handwritten digit images (8×8 pixel `digits` dataset, 1797 samples, 10 classes, used here as a lightweight, fully offline stand-in for the larger 28×28 MNIST benchmark, since both pose the identical multi-class image classification task).

Inference: Each image is a 64-dimensional pixel-intensity vector; the high dimensionality relative to sample count is the classic motivation for PCA-based dimensionality reduction before applying distance-based classifiers such as k-NN or SVM.

5 Data Preprocessing

Preprocessing needs identified per dataset (implemented in full in the experiments that model them):

- **Missing-value handling** – required for Loan Amount Prediction (149 missing values across columns); Iris, Spambase and `digits` are clean (imputation-free) by construction, and Diabetes is used here in its raw form.
- **Categorical encoding** – required for Loan Amount Prediction (5 categorical features); the other four datasets are purely numeric.
- **Feature standardization** – appropriate for all numeric-feature datasets before applying scale-sensitive algorithms such as KNN, SVM or PCA.

No single preprocessing pipeline is executed in this experiment; each dataset’s specific preprocessing is implemented and evaluated in the experiment where it is modelled (Loan → Experiment 3; Spambase → Experiments 2 and 4).

6 Machine Learning Algorithm

NumPy

Explored array creation, reshaping, and vectorized operations:

```

1 a = np.arange(12).reshape(3, 4)
2 a.sum(), a.mean(), a.std()    # 66, 5.5, 3.452
3 a.T                          # transpose
4 a @ a.T                      # matrix product

```

Inference: NumPy's vectorized operations (`sum`, `mean`, `@` for matrix multiplication) avoid explicit Python loops and are the computational backbone that Pandas and Scikit-learn are built on.

Pandas

Explored `DataFrame` construction, `groupby`, and `describe`:

```
1 df.groupby("category").mean(numeric_only=True)
2 df.describe()
```

Inference: `groupby` is the primary tool for computing per-category statistics, and `describe()` gives a fast statistical summary (count, mean, std, quartiles) that is normally the first step of any EDA.

SciPy

Explored hypothesis testing (`scipy.stats`) and linear algebra (`scipy.linalg`):

```
1 t_stat, p_val = stats.ttest_ind(group_A, group_B)    # t=-0.094, p
               =0.928
2 eigvals, eigvecs = linalg.eig(matrix)                # [5.0, 2.0]
```

Inference: An independent two-sample t -test on the synthetic demo groups gave $p = 0.928$, correctly indicating no significant difference (the two groups were drawn from the same distribution by construction). `scipy.linalg.eig` confirmed the eigenvalues of a simple 2×2 matrix, illustrating SciPy's role as the numerical layer beneath many ML algorithms (e.g. PCA relies on eigendecomposition).

Scikit-learn

Explored `SelectKBest` for univariate feature selection:

```
1 SelectKBest(score_func=f_classif, k=2).fit(iris.data, iris.target
   ).scores_
2 # [119.26, 49.16, 1180.16, 960.01]
```

Inference: On the Iris dataset, petal length and petal width (F-scores 1180.16 and 960.01) are far more discriminative between species than sepal length and width (119.26 and 49.16), matching the well-known fact that petal measurements separate Iris species much more cleanly than sepal measurements.

Matplotlib

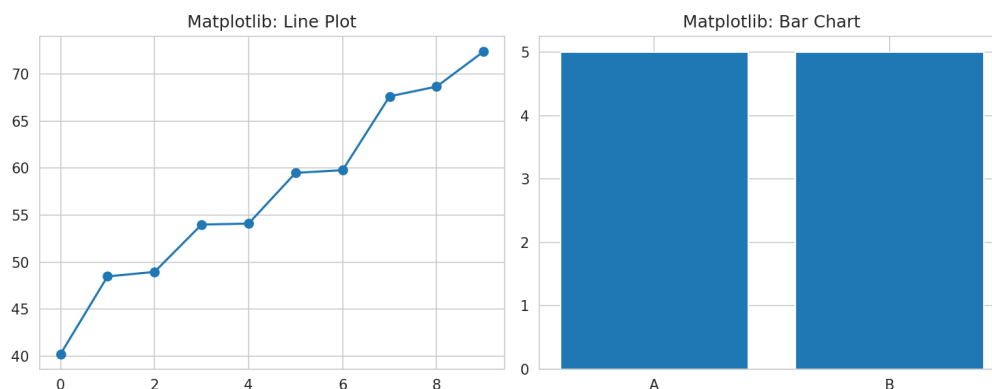


Figure 6: Matplotlib line plot and bar chart generated from the demonstration DataFrame.

Inference: `plt.subplots` is the standard way to lay out multiple axes in one figure, used throughout Experiments 2–4 for combining several related plots into a single saved image.

Summary: Dataset → ML Task → Feature Selection → Algorithm

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Supervised – Multi-class Classification	ANOVA F-test / SelectKBest (all 4 features informative; petal features dominate)	Logistic Regression / KNN / SVM
Loan Amount Prediction	Supervised – Regression (continuous target)	Correlation analysis + domain-driven selection (income, credit history)	Linear / Ridge / Lasso / Elastic Net Regression
Predicting Diabetes	Supervised – Binary Classification	ANOVA F-test (top: Glucose, BMI, Age, Pregnancies)	Logistic Regression / Random Forest / SVM
Classification of Email Spam	Supervised – Binary Classification	Correlation with label + variance threshold (57 numeric features)	Naive Bayes / KNN / Logistic Regression / SVM
Handwritten Character Recognition / MNIST	Supervised – Multi-class Classification (10 classes)	PCA (dimensionality reduction on pixel features)	CNN / SVM (RBF kernel) / Random Forest / k-NN

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core Ultra 9, NVIDIA RTX 5060

8 Hyperparameter Selection

Not applicable. This experiment surveys library functions and five reference datasets to identify appropriate tasks, feature-selection techniques and algorithms; it does not train or tune a single model, so no hyperparameter search was performed. Hyperparameter tuning for these same datasets and algorithm families is carried out in Experiments 2–4.

9 Results

Not applicable. No model is trained or evaluated in this experiment, so there are no accuracy/precision/recall-style results to report. The closest analogue is the dataset $\rightarrow$ task $\rightarrow$ technique $\rightarrow$ algorithm summary table in Section 6 (Machine Learning Algorithm).

10 Result Visualization

Not applicable in the usual sense, since there is no trained model to visualize results for. This experiment’s visualizations are exploratory rather than post-model, and are presented in Section 4 (Exploratory Data Analysis) instead.

11 Discussion

Across all five datasets, the same general machine learning workflow recurs:

1. **Loading the dataset** – from CSV / library loaders (`sklearn.datasets`).
2. **EDA and visualization** – histograms, scatter plots, boxplots, count plots, as shown in Section 4 for each dataset.
3. **Data preprocessing** – missing value handling (Loan, Diabetes), encoding categorical variables (Loan), and standardization (all numeric-feature datasets).
4. **Feature selection** – `SelectKBest`/ANOVA F-test for the classification datasets, correlation analysis for the regression dataset, and PCA for the high-dimensional image dataset.
5. **Data splitting** – stratified train/test (and cross-validation) splits, as applied throughout Experiments 2–4.
6. **Performance evaluation** – accuracy/precision/recall/F1/ROC-AUC for classification tasks; MAE/MSE/RMSE/ R^2 for the regression task.

Every later experiment in this course follows these same six steps, differing only in which preprocessing, feature-selection and algorithm choices apply to its specific dataset.

12 Conclusion

This experiment built working familiarity with the core Python data-science stack (NumPy, Pandas, SciPy, Scikit-learn, Matplotlib) and produced a task/technique/algorithm taxonomy for five reference datasets spanning multi-class classification, regression, binary classification and high-dimensional image classification. No single model was trained here; instead, the experiment built the library familiarity, task-identification skills, and reusable six-step workflow that Experiments 2–4 use directly.

This experiment also reinforced the following learning outcomes:

- Gained hands-on familiarity with NumPy, Pandas, SciPy, Scikit-learn and Matplotlib as the core Python data-science stack.
- Learned to distinguish supervised classification, supervised regression and image-based classification tasks from a dataset's feature types and target variable.
- Understood how feature-selection technique choice depends on data type (ANOVA F-test for continuous features vs. a classification target; PCA for high-dimensional pixel data).
- Recognized the machine learning workflow (load → EDA → preprocess → select features → split → evaluate) as a common pattern reused across all subsequent experiments in this laboratory course.

13 References

1. NumPy Developers, <https://numpy.org/doc/>
2. Pandas Developers, <https://pandas.pydata.org/docs/>
3. SciPy Developers, <https://docs.scipy.org/doc/scipy/>
4. Scikit-learn Developers, <https://scikit-learn.org/stable/>
5. Matplotlib Developers, <https://matplotlib.org/stable/>
6. UCI Machine Learning Repository, <https://archive.ics.uci.edu/>
7. Kaggle Datasets, <https://www.kaggle.com/datasets>